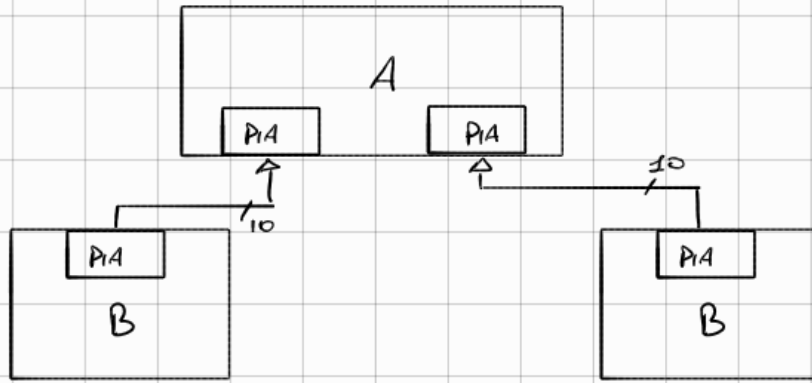
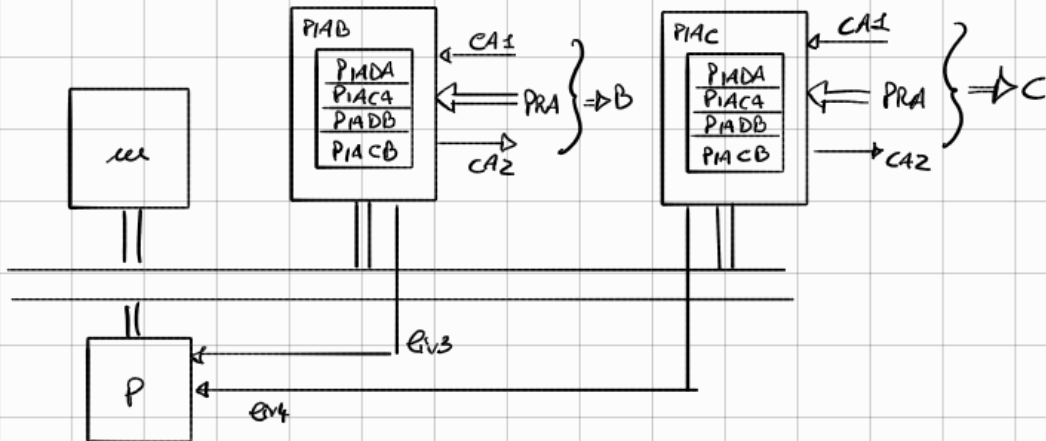


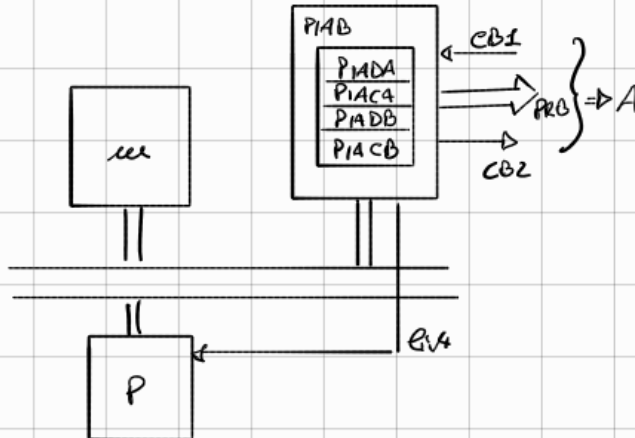
A1)



A:

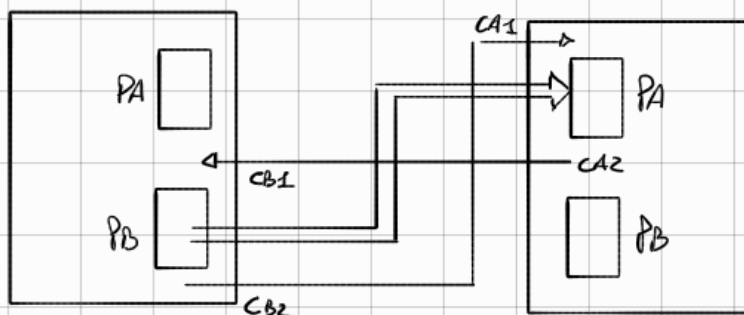


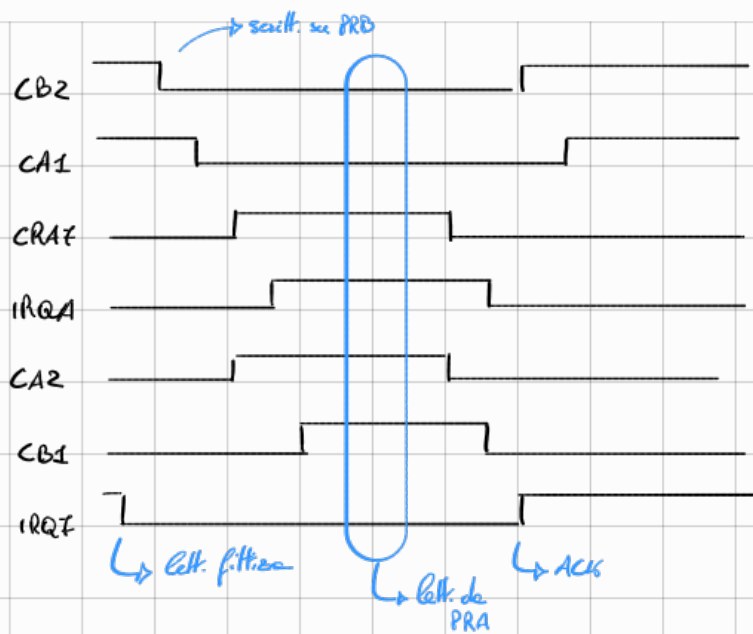
B/C:



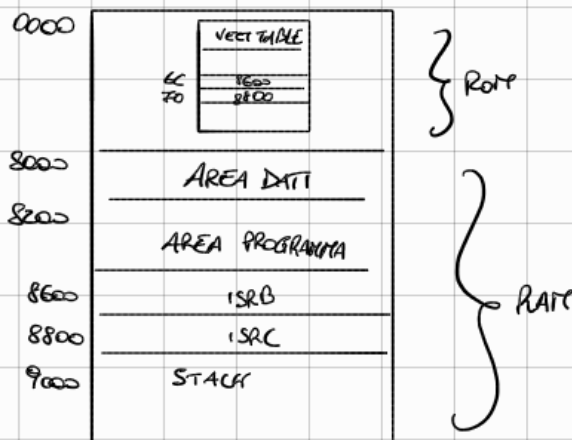
A2)

Di seguito, si illustra il diagramma temporale rappresentante il protocollo di comunicazione del PIA in mod. 100. La config presa in considerazione è





A3)



A4)

```
main() {
    initPias()
    SR = SR AND $D8FF
    :
    loop
}
```

```
initPias() {
    piaB.ca = 0
    piaB.da = 0
    piaB.ca = %001000101
    piaC.ca = 0
    piaC.da = 0
    piaC.ca = %001000101
```

```
ISR B() {
    pendingB = 1
    if (MSG received < 2N) { // phase 1
        if (sender == None) {
            if (TAS(lock)) {
```

```

if (sender == None) {
    sender = B
    if (MSGreceived == 0) {
        phase1first = B
    }
}
lock = 0
} else {
    RTE
}
}

if (pendingB == 1 and sender == B) {
    buffB[k * countMSGB + currchB] = pieB. da
    currchB ++
    pendingB = 0
    if (currchB == K) {
        countMSGB ++
        currchB = 0
        MSGreceived ++
        if (MSGreceived == N) {
            sender = C → passo formalmente il possesso del canale a C
            if (pendingC == 1) {
                readPos = currchC
                currchC ++
                pendingC = 0
                buffC[readPos] = pieC. da
            }
        } else if (MSGreceived == 2 * N) {
            sender = None
        }
    }
}

```

```

    }
}
} else {
    // phase 2
    if (phase1first == B)
        buffB[16 * N + curorchB] = pieB.da // scelta di progetto → i messaggi di phase2 si concatenano
        curorchB++ // in memoria a quelli di phase1

    pendingB = 0
    if (curorchB == 16) {
        pieB.ca = pieB.ca AND $FE // ho ricevuto tutto il mex
        // → bisogna terminare
        // trasmissione
    }
} else {
    // phase1first == C
    if (sender == None) {
        if (TAS(lock)) {
            if (sender == None) {
                sender = B
            }
            lock = 0
        } else { RTE }
    }

    if (pendingB == 1 and sender == B) {
        buffB[16 * N + curorchB] = pieB.da
        curorchB++
        pendingB = 0
        if (curorchB == 16) {
            pieB.ca = pieB.ca AND $FE
            sender = C
            if (pendingC == 1) {
                readPos = curorchC + 16 * N
                curorchC++
                pendingC = 0
            }
        }
    }
}

```

$buffC[readPos] = piaC.da$

}

}

}

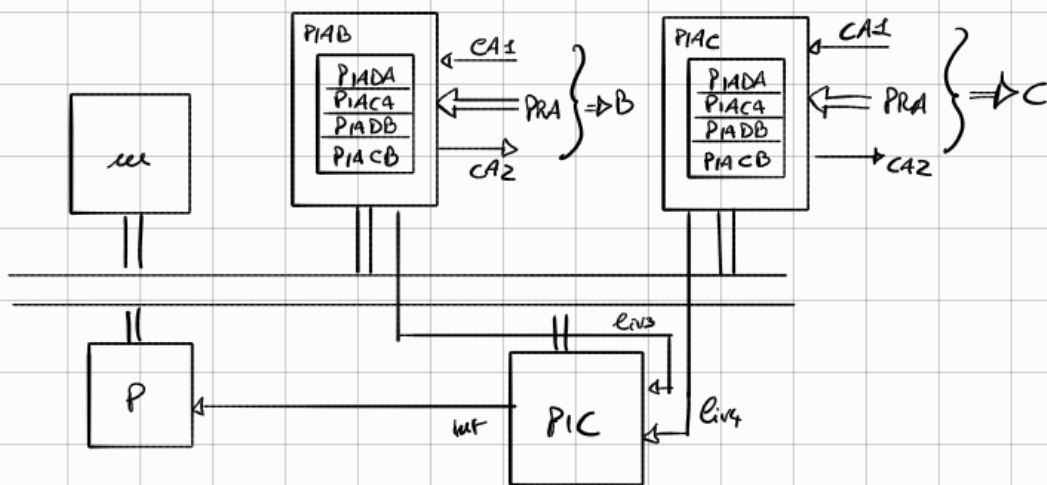
}

}

→ ISC è speculatore

A5) Repo.

B2)



```

MAIN
...
JSR INITPIC
...
LOOP BRA LOOP

```

```

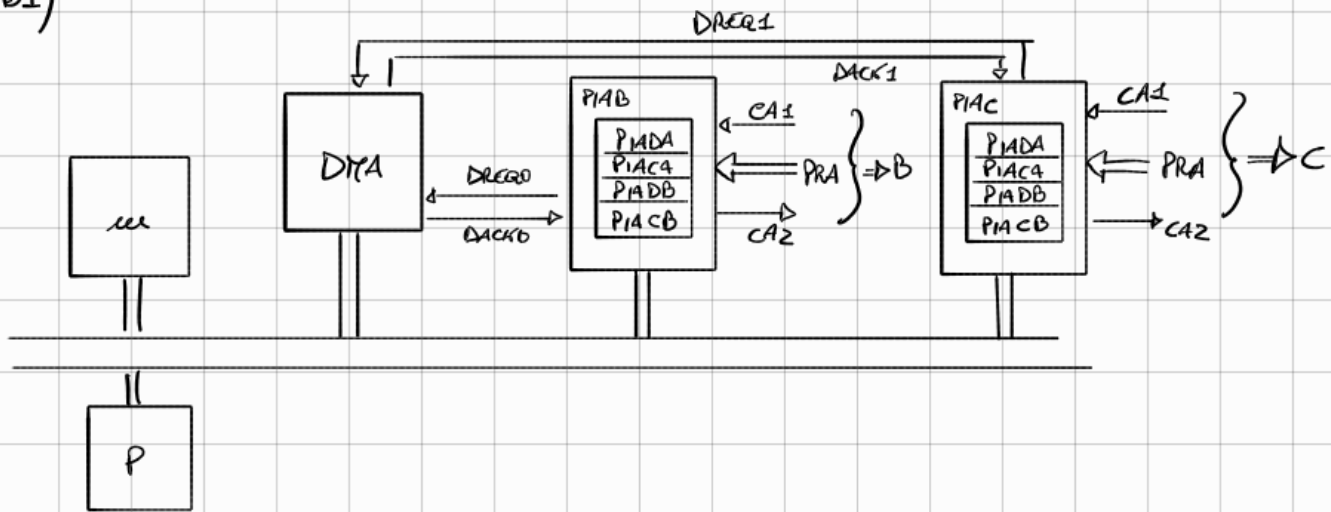
INITPIC  MOVEA.L #PIC, A0
         MOVE.B #540, 1(A0)
         MOVE.B #0, (A0)
         MOVE.B #0, 1(A0)
         RTS

```



B3) Il sistema è stato progettato in maniera tale che non andrebbero apportate modifiche.

B1)



*AREA DATI

DMA EQU \$2010

CADDR0 " 0

CCOUNT0 " 1

CCADDR " 2

CCOUNT1 " 3

MODE " 11

CNTRL " 8

REQUEST " 9

INITDMA

MOVE.L #DMA, A0

MOVE.W #BUFFB, CADDR0(A0)

MOVE.W #BUFFC, CADDR1(A0)

MOVE.B #K, CCOUNT0(A0)

MOVE.B #K, CCOUNT1(A0)

MOVE.B #\$18, MODE(A0)

MOVE.B #\$80, CNTRL(A0)

*

RTS

* la ASIT è necessario fare
MOVE.B #\$08, REQUEST(A0)

MAIN

JSR INITDMA

LOOP BRA LOOP

ORG \$8900

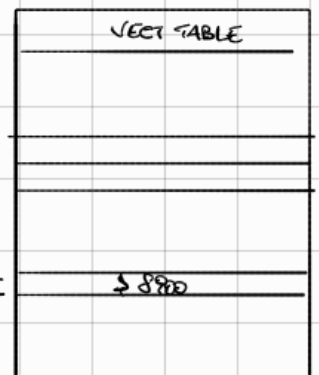
INTF *update counter, ecc

*init di phase 1 first

RTIE

\$6C
\$70

\$7C



Passando a questa architettura non è possibile rispettare più i vincoli riguardanti le fasi in cui si richiede che i messaggi non siano sovrapposti.